

Практическое занятие №3

Обучение нейронной сети. Метод обратного распространения ошибки. Стохастический градиентный спуск.

Нейронная сеть по своей сути представляет собой группу связанных между собой нейронов. Простая нейронная сеть выглядит так, как показано на рис. 5.8.

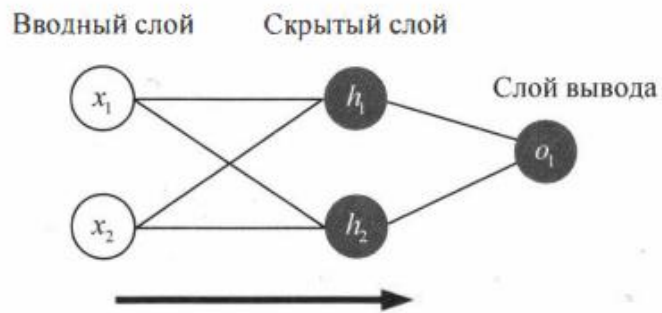


Рис. 5.8. Схема однослойной нейронной сети прямого распространения

На данном рисунке представлена нейронная сеть с тремя слоями. Один вводный слой, на вводном слое сети два входа — x_1 и x_2 . Один скрытый слой, на скрытом слое два нейтрона — h_1 и h_2 . Один слой вывода, на нем находится один нейрон — o_1 .

Скрытым слоем называется любой слой между входным слоем и слоем вывода, которые являются первым и последним слоями соответственно. Скрытых слоев может быть несколько. Обратите внимание на то, что входные данные для o_1 являются результатами вывода h_1 и h_2 . По такому принципу и строятся нейронные сети. Направление передачи сигналов слева направо. Такие сети относятся к сетям прямого распространения (feed forward).

Давайте используем продемонстрированную выше схему сети и представим, что в качестве ввода будет использовано значение $x = [2, 3]$, нейроны имеют вес $w = [0, 1]$ и одинаковое смещение $b = 0$. В качестве функции активации будет сигмоида. Схематично сеть с такими параметрами представлена на рис. 5.9.

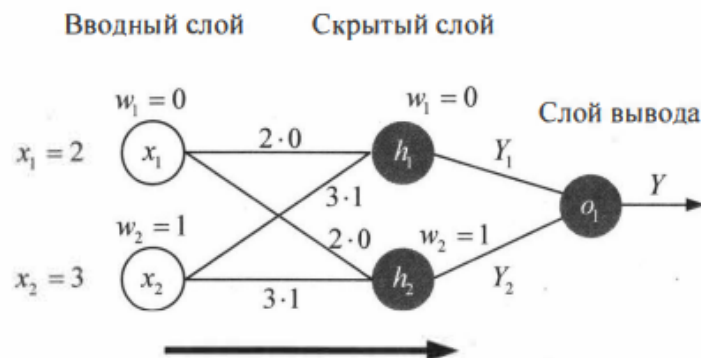


Рис. 5.9. Пример однослойной нейронной сети прямого распространения

Подсчитаем, какие результаты выдаст сеть с такими параметрами. Нейроны h_1 и h_2 на свои входы получают одинаковые значения: $x_1 = 2$, $x_2 = 3$. На выходе они сформируют сигналы Y_1 и Y_2 , которые примут следующие значения:

$$Y_1 = f(x_1 w_1 + x_2 w_2 + b) = f(2 \cdot 0 + 3 \cdot 1 + 0) = f(3);$$

$$Y_2 = f(x_1 w_1 + x_2 w_2 + b) = f(2 \cdot 0 + 3 \cdot 1 + 0) = f(3),$$

где f — функция активации.

Далее выходы из нейронов скрытого слоя Y_1 и Y_2 станут входными сигналами на нейрон слоя вывода o_1 . Затем нейрон o_1 сформирует общий выходной сигнал из сети Y :

$$Y = f(Y_1 w_1 + Y_2 w_2 + b) = f(Y_1 \cdot 0 + Y_2 \cdot 1 + 0).$$

Программный код, реализующий данную сеть, представлен в листинге 5.4.

Листинг 5.4

```
# Модуль Net1
import numpy as np
```

```

# функция активации
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Описание класса Нейрон
class Neuron:
    def __init__(self, weights, bias):
        self.weights = weights
        self.bias = bias

    def feedforward(self, inputs):
        total = np.dot(self.weights, inputs) + self.bias
        return sigmoid(total)

# Описание класса Нейронная сеть из 3 слоев
class OurNeuralNetwork:

    def __init__(self):
        weights = np.array([0, 1]) # веса (одинаковы для всех нейронов)
        bias = 0 # смещение (одинаково для всех нейронов)

        # формируем сеть из трех нейронов
        self.h1 = Neuron(weights, bias)
        self.h2 = Neuron(weights, bias)
        self.o1 = Neuron(weights, bias)

    def feedforward(self, x):
        out_h1 = self.h1.feedforward(x) # Формируем выход Y1 из нейрона h1
        out_h2 = self.h2.feedforward(x) # Формируем выход Y2 из нейрона h2
        out_o1 = self.o1.feedforward(np.array([out_h1, out_h2])) # Формируем выход Y
                                                                    # из нейрона o1

        return out_o1

network = OurNeuralNetwork() # Создаем объект СЕТЬ из класса "Наша нейронная сеть"
x = np.array([2, 3]) # формируем входные параметры для сети X1=2, X2=3
print("Y=", network.feedforward(x)) # Передаем входы в сеть и получаем результат

```

Данная программа содержит несколько блоков.

1. Подключение библиотеки для работы с математическими функциями NumPy:

```
import numpy as np
```

Из этой библиотеки здесь используются следующие функции:

- `array()` — формирование одномерного массива;
- `dot()` — определение скалярного произведения элементов массива.

2. Описание класса `Neuron` (нейрон):

```
class Neuron:
```


3. Описание класса `OurNeuralNetwork` (нейронная сеть из трех слоев):

```
class OurNeuralNetwork:
```

4. Создание объекта `СЕТЬ` из класса "Наша нейронная сеть":

```
network = OurNeuralNetwork()
```

5. Формирование входных параметров для сети и передача их в нашу новоиспеченную сеть:

```
x = np.array([2, 3])  
print(network.feedforward(x))
```

Меняя входные значения x , мы будем получать итоговый ответ Y от сети. Запустив программу на выполнение, мы получим результат, как на рис. 5.10.

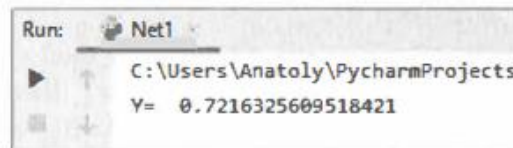


Рис. 5.10. Итог работы однослойной нейронной сети прямого распространения

Обучаем нейронную сеть

В предыдущем разделе мы показали, как из нейронов построить простейшую сеть, и реализовали ее в виде программного кода на Python. Затем на вход сети мы дали набор входных параметров и на выходе получили какой-то ответ. Сеть выполнила свою задачу на наборе тестовых цифр, абсолютно не связанных с реальной действительностью. Попробуем теперь поставить перед нашей сетью близкую к жизни задачу и научить ее находить правильное решение. Для того чтобы понять, как это делается, максимально упростим условия задачи. Попробуем научить нейронную сеть определять пол человека в зависимости от его веса и роста. Для этого сначала нужно сделать замеры этих параметров. Результаты такой работы приведены в табл. 5.2.

Таблица 5.2. Замеры роста и веса людей для подготовки обучающей выборки

Имя	Вес, фунты	Рост, дюймы	Пол
Alice	133	65	Ж
Bob	160	72	М
Charlie	152	70	М
Diana	120	60	Ж

Видно, что мужчины имеют бóльшие рост и вес, чем женщины. Изменим символическое обозначение пола с М и Ж на 0 и 1, т. е. ответ сети 0 будем понимать как "мужчина", а ответ 1 — как "женщина". Также сделаем смещение роста и веса на неко-

торую постоянную величину. Обычно для смещения выбираются средние показатели, мы обозначим следующие величины смещения: рост уменьшим на 135 футов (–135), вес уменьшим на 66 футов (–66). На основе этих данных сформируем обучающую выборку (табл. 5.3).

Таблица 5.3. Обучающая выборка для определения пола человека по его росту и весу

Имя	Вес (– 135)	Рост (–66)	Пол
Alice	–2	–1	1
Bob	25	6	0
Charlie	17	4	0
Diana	–15	–6	1

Перед тренировкой нейронной сети выберем способ оценки того, насколько хорошо сеть справляется с задачами (ошибается или нет). Введем такое понятие, как *потери* (потеря правильного решения). Для данного примера для оценки потери будет использоваться расчет среднеквадратичной ошибки. Не будем углубляться в дебри математики, а рассмотрим, как делается расчет этой ошибки на простом примере. Допустим, мы имеем два значения: заведомо верное решение Y_{true} и предположительное решение Y_{pred} (полученный от сети ответ). Тогда величину ошибки сети E можно подсчитать следующим образом:

$$E = Y_{\text{true}} - Y_{\text{pred}}.$$

Если мы давали сети задание решить нашу задачу многократно (n раз) и для каждого раза делали расчет ошибки (E_i), среднеквадратичную ошибку подсчитаем следующим образом:

$$E_{\text{cp}} = \frac{E_1^2 + E_2^2 + E_3^2 + \dots + E_n^2}{n}.$$

Давайте разберемся с обозначениями в данных формулах:

- n — число рассматриваемых объектов, которое в нашем случае равно 4. Это Alice, Bob, Charlie и Diana;
- Y_{true} — истинное значение правильного ответа (для Alice значение $Y_{\text{true}} = 1$, она женщина);
- Y_{pred} — предполагаемое значение переменной. Это результат вывода сети (мужчина или женщина);
- E_{cp} — средняя квадратичная ошибка.

Из всего вышесказанного важно понять: чем меньше сеть будет ошибаться, тем значение этого показателя будет ниже, т. е. "лучшие предсказания = меньшие потери". Давайте реализуем на Python функцию, которая будет делать расчет этой ошибки. В листинге 5.5 приведен текст соответствующей программы.


```
# расчет среднеквадратической ошибки
def mse_loss(y_true, y_pred):
    # y_true и y_pred являются массивами numpy с одинаковой длиной
    return ((y_true - y_pred) ** 2).mean()
```

Обучение нейронной сети будет заключаться в подборе таких значений параметров нейронов, при которых ошибочность в принятии решений будет минимальной, т. е. нужно свести к минимуму потерю правильных решений. Ясно, что повлиять на предсказания сети можно при помощи изменения весов связей и смещений. Однако каким способом можно минимизировать потери? Рассмотрим это на примере с максимальным упрощением задачи. Оставим в обучающей выборке только одного человека — Alice (табл. 5.4).

Таблица 5.4. Упрощенная обучающая выборка для определения пола человека по его росту и весу

Имя	Вес (–135)	Рост (–66)	Пол
Alice	–2	–1	1

Для Alice потеря правильного ответа будет просто квадратичной ошибкой, которую при значении правильного ответа ($Y_{\text{true}} = 1$) можно подсчитать по следующей формуле:

$$L = (1 - Y_{\text{pred}})^2.$$

Еще один способ понимания потери правильного решения — это представление потери в виде функции от таких параметров сети, как веса связей и смещения (не нужно путать веса связей нейронов с весом наших героев). То есть

$$L = f(W, b).$$

С помощью данной функции можно, меняя веса связей и смещения, найти минимальное значение ошибки. Давайте обозначим индивидуальным индексом каждый вес связи и смещения в рассматриваемой сети (рис. 5.11).

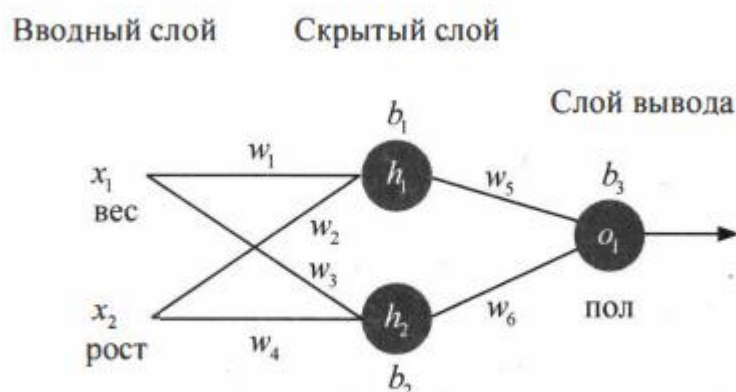


Рис. 5.11. Перечень параметров, влияющих на ошибочность решения нейронной сети

Теперь можно записать в общем виде многовариантную функцию зависимости потери правильного решения от параметров нейронной сети:

$$L = f(w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3).$$

Представим, что нам нужно изменить вес w_1 . В таком случае, как изменится потеря L после внесения поправок в w_1 ? На этот вопрос может ответить частная производная dL/dw_1 . А как ее вычислить? Здесь нужно использовать *систему подсчета частных производных*, или так называемый *метод обратного распространения ошибки* (backprop). Понять суть этого метода, выраженного языком математиков, достаточно сложно. Но если не углубляться в математические рассуждения и положиться на выводы великих математиков, то частную производную (dL/dw_1) можно получить из следующих выражений:

$$\frac{dL}{dw_1} = \frac{dL}{dY_{\text{pred}}} \cdot \frac{Y_{\text{pred}}}{dh_1} \cdot \frac{dh_1}{dw_1};$$

$$\frac{dL}{dY_{\text{pred}}} = -2 \cdot (1 - Y_{\text{pred}});$$

$$\frac{dY_{\text{pred}}}{dh_1} = w_5 \cdot f(h_1 w_5 + h_2 w_6 + b_3);$$

$$\frac{dh_1}{dw_1} = x_1 \cdot f(x_1 w_1 + x_2 w_2 + b_1).$$

Без глубокого знания основ высшей математики в этих методах и формулах легко запутаться. Поэтому мы не будем в них углубляться, а для лучшего понимания принципа их работы рассмотрим простой пример. Будем по-прежнему использовать сведения только об одном человеке с именем Alice (см. табл. 5.4). Посмотрим, что получится, если мы сведения об Alice прогоним через нашу сеть при начально заданных параметрах: все веса связей в сети $w_i = 1$, все смещения $b_i = 0$. Работая с обычным калькулятором, получим следующие результаты:

$$h_1 = f(x_1 w_1 + x_2 w_2 + b_1) = f(1 \cdot (-2) + 1 \cdot (-1) + 0) = f(-2 - 3 + 0) = f(-3) = 0,0474;$$

$$h_2 = f(x_1 w_3 + x_2 w_4 + b_2) = f(1 \cdot (-2) + 1 \cdot (-1) + 0) = f(-2 - 3 + 0) = f(-3) = 0,0474;$$

$$a_1 = f(h_1 w_5 + h_2 w_6 + b_3) = f(0,0474 + 0,0474 + 0) = 0,524.$$

ПРИМЕЧАНИЕ

Следует напомнить, что в качестве функции активации $f(x)$ в приведенных выше выражениях использовалась сигмоида. Например, значение $f(-3) = 0,0474$ получено с применением именно этой функции. Это можно легко проверить, если использовать следующий фрагмент кода программы:


```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
print('h1= ', sigmoid(-3))
```

Результат работы фрагмента этого программного кода: $h1 = 0.04742587317756678$.

Итак, расчеты с калькулятором показали, что на выходе из нейронной сети мы получим результат:

$$Y_{\text{pred}} = 0,524.$$

Однако это дает нам слабое представление о том, является Alice мужчиной или женщиной, поскольку значение предполагаемого решения $Y_{\text{pred}} = 0,524$ находится почти в середине интервала 0–1 (0 — мужчина, 1 — женщина).

Давайте теперь подсчитаем частную производную dL/dw_1 . Сделаем соответствующие расчеты на основе данных нашей Alice. Получим следующие результаты:

$$dL/dY_{\text{pred}} = -2 \cdot (1 - Y_{\text{pred}}) = -2 \cdot (1 - 0,524) = -0,952;$$

$$dY_{\text{pred}}/dh_1 = w_5 \cdot f(h_1 w_5 + h_2 w_6 + b_3) = 1 \cdot f(0,0474 + 0,0474 + 0) = 0,249;$$

$$dh_1/dw_1 = x_1 \cdot f(x_1 w_1 + x_2 w_2 + b_1) = -2 \cdot f(-2 + (-1) + 0) = -0,0904;$$

$$dL/dw_1 = -0,952 \cdot 0,249 \cdot (-0,0904) = 0,0214.$$

Результаты получили, а что с ними делать дальше? А дальше нужно использовать алгоритм оптимизации под названием "стохастический градиентный спуск", который скажет нам, как именно поменять веса связей и смещения для минимизации потерь. Суть этого метода отражается в следующем выражении:

$$w_1 = w_1 - (\eta \cdot dL/dw_1),$$

где η — константа, которую можно трактовать, как скорость обучения.

Все, что мы делаем, так это вычитаем dL/dw_1 из w_1 . При этом:

- если dL/dw_1 имеет положительное значение, то w_1 уменьшится, что приведет к уменьшению потери L ;
- если dL/dw_1 имеет отрицательное значение, то w_1 увеличится, что приведет к увеличению потери L .

Если мы правильно применим эти действия на каждый вес связи и на каждое смещение в сети, то добьемся снижения потерь L , а тем самым и улучшения эффективности работы сети (она станет меньше ошибаться). То есть сеть будет постепенно обучаться и принимать все более правильные решения.

В нашем конкретном случае с Alice мы с помощью обыкновенного калькулятора получили следующее значение $dL/dw_1 = 0,0214$ (при весе $w_1 = 1$). Поскольку рассчитанное значение частной производной оказалось положительным, то значение

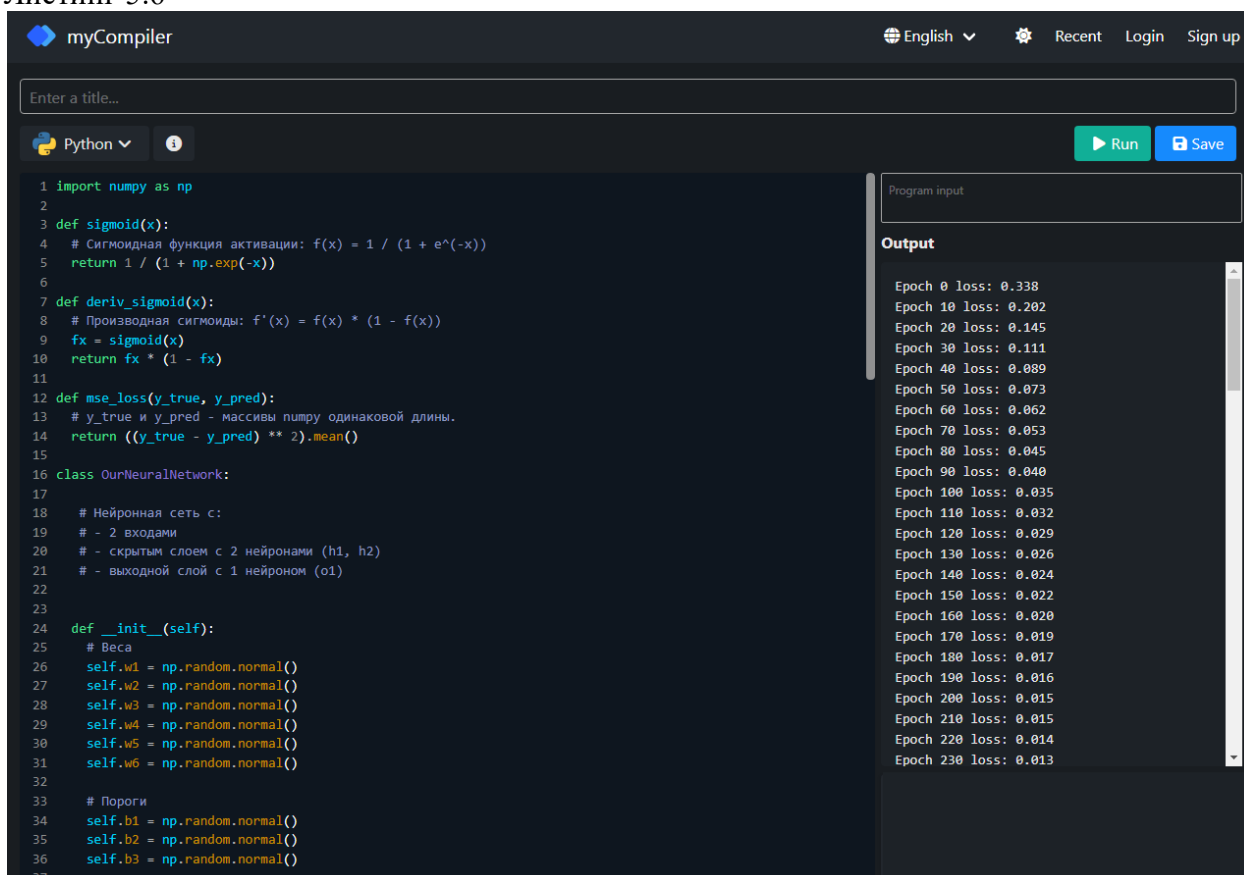
веса w_i возрастет, а значит, будет улучшена эффективность работы сети. А на какую величину возрастет или уменьшится вес w_i , определит коэффициент скорости обучения η . Чем больше будет значение данного коэффициента, тем быстрее сеть будет обучаться, и наоборот. Вспомните пример с кенгуром, η — это как раз длина прыжка кенгуру при движении к водопою. Этот прыжок не должен быть очень маленьким, но и не слишком большим. Оптимизация данного параметра в процессе обучения сети — отдельный вопрос. А пока сформулируем итог: что же нужно делать для того, чтобы сеть смогла обучиться выполнять заданные ей действия?

1. Выбираем один пункт из набора данных (обучающей выборки).
2. Подсчитываем все частные производные потери по весу или смещению.
3. Выполняем обновления каждого веса и смещения.
4. Возвращаемся к первому пункту.

Давайте посмотрим, как это работает на практике. Реализуем этот укрупненный алгоритм обучения сети на примере задачи распознавания людей по их росту и весу.

В листинге 5.6 приведен полный код соответствующей программы на Python.

Листинг 5.6



The screenshot shows a web-based Python IDE named 'myCompiler'. The interface includes a top bar with 'English', 'Recent', 'Login', and 'Sign up' links. Below the bar is a text input field for a title and a toolbar with 'Python' and 'Run' buttons. The main area contains a Python script for a neural network. The script defines a sigmoid function, its derivative, a mean squared error (MSE) loss function, and a neural network class named 'OurNeuralNetwork'. The class has an initialization method that sets weights and biases to random normal values. The output panel on the right shows the training progress over 230 epochs, with the loss decreasing from 0.338 to 0.013.

```
1 import numpy as np
2
3 def sigmoid(x):
4     # Сигмоидная функция активации: f(x) = 1 / (1 + e^(-x))
5     return 1 / (1 + np.exp(-x))
6
7 def deriv_sigmoid(x):
8     # Производная сигмоиды: f'(x) = f(x) * (1 - f(x))
9     fx = sigmoid(x)
10    return fx * (1 - fx)
11
12 def mse_loss(y_true, y_pred):
13    # y_true и y_pred - массивы numpy одинаковой длины.
14    return ((y_true - y_pred) ** 2).mean()
15
16 class OurNeuralNetwork:
17
18    # Нейронная сеть с:
19    # - 2 входами
20    # - скрытым слоем с 2 нейронами (h1, h2)
21    # - выходной слой с 1 нейроном (o1)
22
23
24    def __init__(self):
25        # Веса
26        self.w1 = np.random.normal()
27        self.w2 = np.random.normal()
28        self.w3 = np.random.normal()
29        self.w4 = np.random.normal()
30        self.w5 = np.random.normal()
31        self.w6 = np.random.normal()
32
33        # Пороги
34        self.b1 = np.random.normal()
35        self.b2 = np.random.normal()
36        self.b3 = np.random.normal()
37
```

Output

```
Epoch 0 loss: 0.338
Epoch 10 loss: 0.202
Epoch 20 loss: 0.145
Epoch 30 loss: 0.111
Epoch 40 loss: 0.089
Epoch 50 loss: 0.073
Epoch 60 loss: 0.062
Epoch 70 loss: 0.053
Epoch 80 loss: 0.045
Epoch 90 loss: 0.040
Epoch 100 loss: 0.035
Epoch 110 loss: 0.032
Epoch 120 loss: 0.029
Epoch 130 loss: 0.026
Epoch 140 loss: 0.024
Epoch 150 loss: 0.022
Epoch 160 loss: 0.020
Epoch 170 loss: 0.019
Epoch 180 loss: 0.017
Epoch 190 loss: 0.016
Epoch 200 loss: 0.015
Epoch 210 loss: 0.015
Epoch 220 loss: 0.014
Epoch 230 loss: 0.013
```

```

38 def feedforward(self, x):
39     # x is a numpy array with 2 elements.
40     h1 = sigmoid(self.w1 * x[0] + self.w2 * x[1] + self.b1)
41     h2 = sigmoid(self.w3 * x[0] + self.w4 * x[1] + self.b2)
42     o1 = sigmoid(self.w5 * h1 + self.w6 * h2 + self.b3)
43     return o1
44
45 def train(self, data, all_y_trues):
46     """
47     - data - массив numpy (n x 2) numpy, n = К-во наблюдений в наборе.
48     - all_y_trues - массив numpy с n элементами.
49     Элементы all_y_trues соответствуют наблюдениям в data.
50     """
51     learn_rate = 0.1
52     epochs = 1000 # сколько раз пройти по всему набору данных
53
54     for epoch in range(epochs):
55         for x, y_true in zip(data, all_y_trues):
56             # --- Прямой проход (эти значения нам понадобятся позже)
57             sum_h1 = self.w1 * x[0] + self.w2 * x[1] + self.b1
58             h1 = sigmoid(sum_h1)
59
60             sum_h2 = self.w3 * x[0] + self.w4 * x[1] + self.b2
61             h2 = sigmoid(sum_h2)
62
63             sum_o1 = self.w5 * h1 + self.w6 * h2 + self.b3
64             o1 = sigmoid(sum_o1)
65             y_pred = o1
66
67             # --- Считаем частные производные.
68             # --- Имена: d_L_d_w1 = "частная производная L по w1"
69             d_L_d_ypred = -2 * (y_true - y_pred)
70
71             # Нейрон o1
72             d_ypred_d_w5 = h1 * deriv_sigmoid(sum_o1)
73             d_ypred_d_w6 = h2 * deriv_sigmoid(sum_o1)
74             d_ypred_d_b3 = deriv_sigmoid(sum_o1)

```

Program input

Output

```

Epoch 240 loss: 0.012
Epoch 250 loss: 0.012
Epoch 260 loss: 0.011
Epoch 270 loss: 0.011
Epoch 280 loss: 0.010
Epoch 290 loss: 0.010
Epoch 300 loss: 0.009
Epoch 310 loss: 0.009
Epoch 320 loss: 0.009
Epoch 330 loss: 0.008
Epoch 340 loss: 0.008
Epoch 350 loss: 0.008
Epoch 360 loss: 0.008
Epoch 370 loss: 0.007
Epoch 380 loss: 0.007
Epoch 390 loss: 0.007
Epoch 400 loss: 0.007
Epoch 410 loss: 0.007
Epoch 420 loss: 0.006
Epoch 430 loss: 0.006
Epoch 440 loss: 0.006
Epoch 450 loss: 0.006
Epoch 460 loss: 0.006
Epoch 470 loss: 0.006

```

```

75
76     d_ypred_d_h1 = self.w5 * deriv_sigmoid(sum_o1)
77     d_ypred_d_h2 = self.w6 * deriv_sigmoid(sum_o1)
78
79     # Нейрон h1
80     d_h1_d_w1 = x[0] * deriv_sigmoid(sum_h1)
81     d_h1_d_w2 = x[1] * deriv_sigmoid(sum_h1)
82     d_h1_d_b1 = deriv_sigmoid(sum_h1)
83
84     # Нейрон h2
85     d_h2_d_w3 = x[0] * deriv_sigmoid(sum_h2)
86     d_h2_d_w4 = x[1] * deriv_sigmoid(sum_h2)
87     d_h2_d_b2 = deriv_sigmoid(sum_h2)
88
89     # --- Обновляем веса и пороги
90     # Нейрон h1
91     self.w1 -= learn_rate * d_L_d_ypred * d_ypred_d_h1 * d_h1_d_w1
92     self.w2 -= learn_rate * d_L_d_ypred * d_ypred_d_h1 * d_h1_d_w2
93     self.b1 -= learn_rate * d_L_d_ypred * d_ypred_d_h1 * d_h1_d_b1
94
95     # Нейрон h2
96     self.w3 -= learn_rate * d_L_d_ypred * d_ypred_d_h2 * d_h2_d_w3
97     self.w4 -= learn_rate * d_L_d_ypred * d_ypred_d_h2 * d_h2_d_w4
98     self.b2 -= learn_rate * d_L_d_ypred * d_ypred_d_h2 * d_h2_d_b2
99
100    # Нейрон o1
101    self.w5 -= learn_rate * d_L_d_ypred * d_ypred_d_w5
102    self.w6 -= learn_rate * d_L_d_ypred * d_ypred_d_w6
103    self.b3 -= learn_rate * d_L_d_ypred * d_ypred_d_b3
104
105    # --- Считаем полные потери в конце каждой эпохи
106    if epoch % 10 == 0:
107        y_preds = np.apply_along_axis(self.feedforward, 1, data)
108        loss = mse_loss(all_y_trues, y_preds)
109        print("Epoch %d loss: %.3f" % (epoch, loss))
110

```

Program input

Output

```

Epoch 480 loss: 0.005
Epoch 490 loss: 0.005
Epoch 500 loss: 0.005
Epoch 510 loss: 0.005
Epoch 520 loss: 0.005
Epoch 530 loss: 0.005
Epoch 540 loss: 0.005
Epoch 550 loss: 0.005
Epoch 560 loss: 0.005
Epoch 570 loss: 0.004
Epoch 580 loss: 0.004
Epoch 590 loss: 0.004
Epoch 600 loss: 0.004
Epoch 610 loss: 0.004
Epoch 620 loss: 0.004
Epoch 630 loss: 0.004
Epoch 640 loss: 0.004
Epoch 650 loss: 0.004
Epoch 660 loss: 0.004
Epoch 670 loss: 0.004
Epoch 680 loss: 0.004
Epoch 690 loss: 0.004
Epoch 700 loss: 0.004
Epoch 710 loss: 0.003
Epoch 720 loss: 0.003

```

```

111 # Определим набор данных
112 data = np.array([
113     [-2, -1], # Алиса
114     [25, 6], # Боб
115     [17, 4], # Чарли
116     [-15, -6], # Диана
117 ])
118 all_y_trues = np.array([
119     1, # Алиса
120     0, # Боб
121     0, # Чарли
122     1, # Диана
123 ])
124
125 # Обучаем нашу нейронную сеть!
126 network = OurNeuralNetwork()
127 network.train(data, all_y_trues)

```

```

Epoch 920 loss: 0.003
Epoch 930 loss: 0.003
Epoch 940 loss: 0.003
Epoch 950 loss: 0.002
Epoch 960 loss: 0.002
Epoch 970 loss: 0.002
Epoch 980 loss: 0.002
Epoch 990 loss: 0.002

```

[Execution complete with exit code 0]

Рассмотрим основные блоки данной программы.

1. В первой строке подключаем библиотеку стандартных математических функций NumPy.

2. Определяются следующие функции:

- `sigmoid()` — функция активации;
- функция расчета производной от `sigmoid`;
- функция расчета среднеквадратичной ошибки.

3. Создается класс `OurNeuralNetwork`, реализующий нейронную сеть, изображенную на рис. 5.1.

4. Внутри класса `OurNeuralNetwork` создается функция, которая реализует алгоритм обучения нашей нейронной сети. В этой функции мы задаем скорость и количество циклов обучения:

```
learn_rate = 0.1  
epochs = 1000
```

5. Формируем массив обучающей выборки `data` (вес и рост наших героев см. в табл. 5.3).

6. Формируем массив правильных ответов `all_y_trues` (пол наших героев см. в табл. 5.3).

7. Последним шагом запускаем нашу сеть на учебу (тренируем искать правильные ответы):

```
network = OurNeuralNetwork()  
network.train(data, all_y_trues)
```

Вот, собственно, и всё. Запускаем нашу программу. Через каждые 10 уроков данная программа будет выводить уровень потерь правильных решений. Если посмотреть на эти результаты, то будет видно, что после каждых десяти циклов обучения сеть делает все меньше и меньше ошибок. Для того чтобы продемонстрировать результаты расчетов, изменим в программе всего одну строку:

```
if epoch % 100 == 0
```

Тогда мы получим уровень потерь правильных решений через каждые 100 циклов обучения (рис. 5.12).

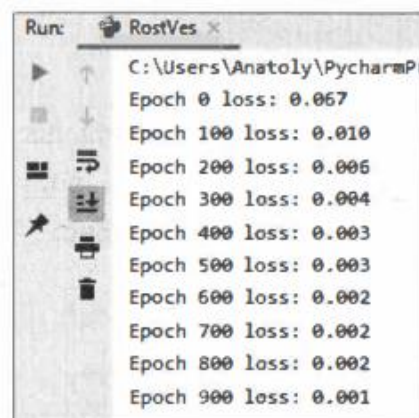


Рис. 5.12. Уменьшение ошибочности решений нейронной сети в процессе обучения

Если теперь на основе выданных программой данных построить график зависимости потерь правильных решений от количества циклов обучения, то получится картина, приведенная на рис. 5.13.

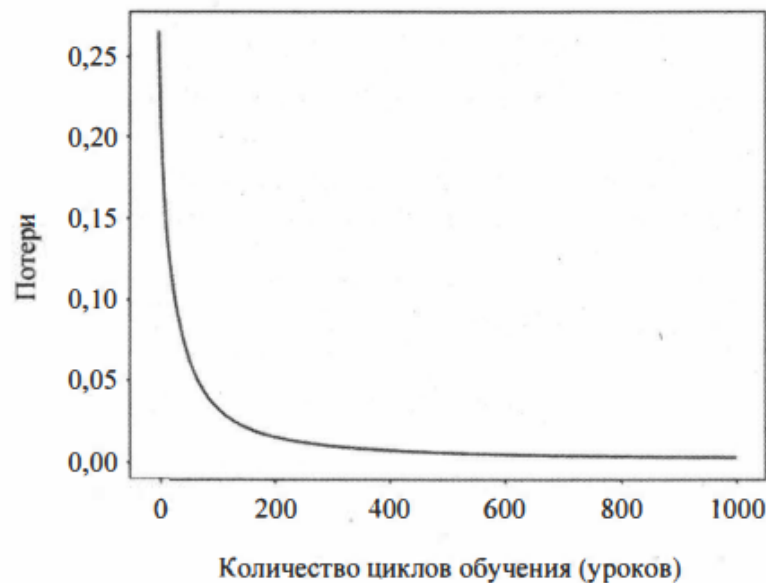


Рис. 5.13. Зависимость ошибочности решений нейронной сети от количества циклов обучения

Видно, что уже после 400 уроков наша сеть получила достаточно опыта для того, чтобы справиться с поставленной задачей.

Теперь дадим нашей обученной сети задание — определить пол людей, которых не было в обучающей выборке. Добавим в программу следующие строки (листинг 5.7).

Листинг 5.7

```
# Делаем предсказания
emily = np.array([-7, -3]) # 128 фунтов, 63 дюйма
frank = np.array([20, 2]) # 155 фунтов, 68 дюймов
print("Emily: %.3f" % network.feedforward(emily))
print("Frank: %.3f" % network.feedforward(frank))
```

Здесь у нас два человека:

- женщина Emily с параметрами: вес — 128 фунтов, рост — 63 дюйма;
- мужчина Frank с параметрами: вес — 15 фунтов, рост — 68 дюймов.

Запустив программу, получим следующий ответ (рис. 5.14).



Рис. 5.14. Результат работы обученной нейронной сети

При обучении нейронной сети мы условились, что правильный ответ 1 будет означать "женщина", а ответ 0 — "мужчина". Для Emily мы получили ответ — 0.996, что близко к единице; значит, это женщина. Для Frank мы получили ответ 0.054, что близко к нулю; значит, это мужчина. То есть обученная сеть справилась с поставленной задачей.

Теперь посмотрим, какие же значения получили веса связей и смещений после завершения процесса обучения. Для этого после строки программы:

```
print("Epoch %d loss: %.3f" % (epoch, loss))
```

добавим следующие строки:

```
print('w1=', self.w1)
print('w2=', self.w2)
print('b1=', self.b1)
print('w3=', self.w3)
print('w4=', self.w4)
print('b2=', self.b2)
print('w5=', self.w5)
print('w6=', self.w6)
print('b3=', self.b3)
```

После такой модификации программа выдаст результат, как на рис. 5.15.

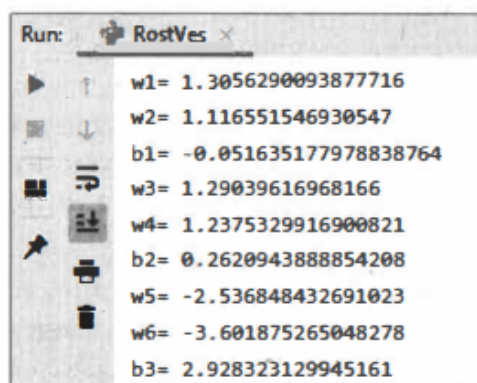


Рис. 5.15. Значения весов связей и смещений после обученной нейронной сети

Это, по сути, итоги обучения. Если эти параметры сохранить, а потом загрузить в структуру сети перед выполнением задания, то сеть справится с поставленной задачей уже без предварительного обучения. То есть нейронная сеть, однажды натренированная на решение какого-либо класса задач, в дальнейшем будет выполнять все порученные задания уже без обучения. Например, если сеть однажды научили распознавать символы, то она будет читать и понимать любые тексты. Если сеть научили выделять лицо человека на фотографии, то она сможет выделить лицо любого человека на любой фотографии. И не только на фотографии, но и на изображении с видеокамеры!